Phase-5 Subgraph Retrieval: What We Have, What We Tested, What We Don't

arxivkg internal analysis

2026-06-29

Abstract

This note inventories the subgraph-retrieval techniques relevant to phase-5 of arxivkg. It describes the *deployed* method in full (induced-pool + ancestor augmentation + Personalized PageRank + structural extraction), reports the head-to-head *bake-off* we ran (induce vs. PCST/Steiner vs. similarity-aware BFS over five topics), and surveys the methods we have *not* tested, separating those that operate on a *given* graph (in-regime) from those that *build* a graph from text (a different paradigm). The central empirical finding is that *membership* and *structure* are separable concerns with different optimal algorithms: citation-centrality (PPR) maximizes recall of canonical landmarks; cosine-prized Steiner trees maximize connectivity but discard those landmarks. Our committed recipe takes the best half of each.

1 The setting, and why it constrains the method space

Phase-5 turns a query vector into a **bounded, on-topic, connected citation subgraph** whose nodes are the "landmark" papers of a field and whose edges are the real citations between them. This subgraph then feeds (a) the phase-6 narrative/compose layer and (b) a lineage-tree visualization (time on x, dendromatic spread on y).

The decisive property of our setting is that **the graph already exists**. We have a real, directed citation graph: 173M edges in Postgres, each edge carrying an LLM-inferred `intent`$\in$\{extends, uses, compares, critiques, motivates, background\} and a `polarity`. This is categorically different from the mainstream "GraphRAG" literature, where the retrieval graph is *synthesized* from a text corpus by an LLM (entity + relation extraction) before retrieval can begin. That distinction partitions the entire method space:

- **Group A — operate on a given graph.** Directly applicable to us; these are *selectors* over our citation edges. The bake-off lives here.
- **Group B — build their own graph from text.** Would have us *discard* the citation structure and re-derive a weaker one with an LLM. Not drop-in competitors for the retrieval path; at most they donate ideas to an *offline* index/browse layer.

A second constraint: phase-6 induces a *small* subgraph ($\sim$400 candidate nodes, $\sim$5,000 edges) and runs everything in Python. We never traverse the 173M-edge graph at query time, so methods whose entire value proposition is native graph-DB traversal (Neo4j-style) buy us nothing here.

2 What we have: the deployed method

The production path (`phases/phase-6/components/{subgraph,pagerank,recency}.py`) is a four-stage pipeline. No LLM and no embedding model run inside it — the caller passes a precomputed query vector so VRAM can be sequenced (embed $\rightarrow$ free model $\rightarrow$ load narrator).

Stage 1 — topical pool (vector + lexical)

A hybrid candidate pool: pgvector cosine KNN (now on the *asymmetric* nomic space, raw query, no HyDE — the phase-4 decision) fused by RRF with a title-BM25 list, then a *frontier pull* (year-filtered pure-KNN recency top-up, because the corpus has a 2026 delta the topic tree predates). Result: ∼140–510 nodes depending on topic breadth.

Stage 2 — backward-ancestor augmentation (graph)

The similarity pool is dominated by recent application/survey papers; the *foundational* lineage (DDPM, LDM, score-SDE for diffusion) is cited by nearly the whole pool yet is rarely a top-N embedding neighbour of the query. `augment_with_ancestors` pulls those most-cited one-hop ancestors in, ranked by the number of distinct pool papers that cite them, optionally category-gated so a famous cross-field paper cannot hijack a polysemous query. This is the single most important recall mechanism — it is what makes "Origins" and the main-path spine exist at all.

Stage 3 — induce the subgraph (graph)

`induce_subgraph` keeps every citation edge with *both* endpoints in the pool, collapsing parallel occurrences to one edge carrying the strongest-signal intent (priority order extends > uses > compares > critiques > motivates > background). Inducing *within* the similarity-ranked pool — rather than expanding outward from seeds — is the key trick that avoids the hub-explosion trap (ResNet/Adam have ∼100k citers): the structure inside the on-topic cloud is automatically relevant and bounded.

Stage 4 — rank + structure (Personalized PageRank)

Raw in-degree favours ancient hubs. Instead `rank_landmarks_ppr` (`pagerank.py`) runs PPR over the induced edges, *teleporting* back to the query-similarity weights:

$$r \;=\; (1-\alpha)\,p \;+\; \alpha\,\big(Wr + (\mathbf{1}_{\text{dangling}}^{\top} r)\,p\big), \qquad \alpha = 0.85,\;\; p_a \propto \max(\cos(a,q),\,0).$$

This yields *topic-personalized influence*: a paper accumulates rank by being cited *within this topic's* cloud, so foundational-yet-on-topic papers rise while generic hubs do not. On top of PPR sit three recency-aware refinements: a topic-adaptive *frontier reservation* (newest-cohort slots reserved by similarity, since recent papers have low rank by construction), an *age-normalized* tiebreak (citations-per-year) for the fill, and the `main_path`/`assign_eras`/`tensions` structural extractors that turn the ranked landmarks into a chronological spine, three eras, and comparison/critique edges.

In one line: membership = similarity pool + cited ancestors; ranking = PPR personalized to the query; structure = longest chronological citation chain.

3 What we tested: the bake-off

We held the candidate pool *fixed* (same Stage 1–3 pool, same induced edges, same per-node cosine "prize") and swapped only the Stage-4 **selector**, across five gold topics (`subgraph_bakeoff.py`). Three selectors:

`induce` **(ours).** PPR-rank the pool to m landmarks. Keeps the whole on-topic pool as membership; centrality decides who is a landmark.

`pcst` — **Prize-Collecting Steiner Tree (G-Retriever, `2402.07630`).** Find the connected subtree maximizing $\sum$ prize $-\sum$ edge cost. We put the prize on *only* the top-m relevant nodes (0 elsewhere) so the solver connects them via minimal Steiner *bridges*; edge cost $\approx 0.4\times$ mean prize tunes how many bridges join.

`dsa_bfs` — **similarity-aware BFS (DynaGRAG, `2412.18644`).** BFS outward from the top-cosine seeds, gated by cos ≥ 0.45, bounded depth 4 / fan-out 8.

Topic	Method	n_{cc}	edges	mean cos	recall	note
diffusion	`induce`	20	165	0.487	0.625	best recall, worst connectivity
	`pcst`	43	183	0.630	0.125	best cos/conn, drops canonical
	`dsa_bfs`	38	344	0.587	0.500	middle
transformer	`induce`	28	131	0.481	0.600	
	`pcst`	40	90	0.592	0.400	
	`dsa_bfs`	36	156	0.577	0.400	
testtime	`induce`	35	204	0.522	1.000	full canonical recall
	`pcst`	40	189	0.585	0.667	
	`dsa_bfs`	33	209	0.567	0.500	
ssm	`induce`	27	171	0.435	—	no gold set
	`pcst`	26	116	0.556	—	
	`dsa_bfs`	35	231	0.550	—	
gnn	`induce`	22	146	0.554	—	no gold set
	`pcst`	39	123	0.664	—	
	`dsa_bfs`	34	227	0.646	—	

Table 1: Bake-off ($m \approx 40$). n_{cc}: nodes in the largest connected component (higher = less fragmented). mean cos: average query-cosine of chosen nodes (topical-drift proxy). recall: fraction of the gold canonical set recovered. Green/red/amber mark the per-column best/worst/mid where a gold set exists.

Reading the table

- `induce` **(PPR) wins recall, loses connectivity.** It recovers the canonical landmarks (diffusion 0.625, testtime 1.0) because PPR rewards *citation centrality*, and canonical papers are central by definition. But the top-m by centrality are not all mutually linked, so the induced tree fragments (only 20–35 of 40 in one component).
- `pcst` **wins connectivity and mean-cosine, loses recall — badly.** Because the prize is *cosine*, PCST keeps tightly-connected high-cosine clusters and *drops the canonical landmarks*, which are often *not* top-cosine neighbours of the query (diffusion recall collapses to 0.125). This is the headline finding: **cosine $\neq$ canonical**. Raw KNN recall of canonical papers is only $\sim$0.16; PPR is precisely the fix, and PCST throws it away.
- `dsa_bfs` **is a consistent middle** on both axes with no regime where it wins.
- An *earlier* PCST run with a uniform low edge cost ($\tau = 0.45$) kept the *whole* graph (404/468 nodes) — a degenerate "connect everything" solution. The fix (prize only on the top-m, 0 elsewhere) is what produces the focused $\sim$40-node trees above.

The synthesis we committed to

The two failures are complementary, so the lineage-tree builder (`scratchpad/lineage_tree.py`) uses **each algorithm for what it is good at**:

1. **Membership = PPR landmarks** (recall): take the `induce` landmark set.
2. **Connectivity = PCST with a big prize on *exactly those landmarks*** (0 elsewhere): the Steiner solver adds the minimal bridge papers that join them into one tree (∼36–40 of 40 kept connected).
3. **Parent structure = most-recent-older-cited predecessor** (clean layout): each paper's parent is the in-set paper it cites with the *latest* year ≤ its own (tie: in-degree). This threads flowing left→right chains (VAE → U-Net/GANs → DDPM → Stable Diffusion → frontier) instead of the ancient-hub star that raw in-degree parents produce.

Slogan: **embeddings + PPR decide *who is in*; the graph decides *how it is shaped*.**

4 What we don't have (yet): surveyed but untested

The literature survey (run over 5 of the 27 topics) returned the methods below. They split cleanly along the Group A / Group B line from §1.

4.1 Group A — in-regime (operate on our citation graph)

SimGRAG (`2412.15272`) — **untested, marginal fit.** Aligns a query to a subgraph by graph-*pattern* similarity (a soft subgraph-isomorphism / structural match). Genuinely operates on a given graph, so it *is* in-regime. But our queries are *topical* ("diffusion models"), not *structural* ("find a motif shaped like X"); there is no query-side graph pattern to match against. Predicted payoff: low. Worth at most a cheap probe as a fourth bake-off selector, mainly to be able to say we tried it.

Full G-Retriever objective (`2402.07630`) — **partially tested.** We use its PCST formulation for connectivity, but *not* its downstream graph-encoder + soft-prompt that feeds the subgraph into an LLM via a learned projection. That half is irrelevant to us: our compose layer reads the subgraph as text/citations, not as a learned graph embedding. No reason to add it.

HippoRAG-style PPR (`2405.14831`) — **effectively already adopted.** HippoRAG's core idea — single-step PPR personalized to query-linked nodes over a knowledge graph — *is* what `pagerank.py` implements (the module docstring credits it). The piece we don't replicate is its synonym/entity-linking front-end (Group B; not needed when edges are real citations).

4.2 Group B — out of regime (build a graph from text)

GraphRAG, Microsoft (`2404.16130`). LLM entity/relation extraction → Leiden community detection → hierarchical community *summaries*, queried for global "sense-making". Replaces, not uses, the citation graph. **But** its community-summary idea maps onto our retired phase-5 `community_summary` component and is the one Group B idea worth reviving — as an *offline browse/index artifact*, decoupled from retrieval.

LightRAG (`2410.05779`). Dual-level (local entity + global theme) retrieval over an LLM-extracted KG with incremental updates. Same paradigm mismatch; the "dual-level" framing is already approximated by our pool (local cosine) + ancestor augmentation (global lineage).

RAPTOR (`2401.18059`). Recursive clustering + abstractive summarization into a tree; purely text-derived, uses *no* edges. Orthogonal to citation-subgraph retrieval. Its recursive summary tree could, like GraphRAG, feed an offline topic-browse layer — not the retrieval path.

4.3 Methods we are *not* pursuing, and why

- **GNN re-rankers (GNN-RAG `2405.20139` and kin).** Would require training a GNN over the citation graph; PPR captures most of the centrality signal at zero training cost and is fully interpretable. Not worth the complexity for a retrieval stage.
- **Native graph-DB traversal (Neo4j GDS).** Retired in the store consolidation; our subgraph is small and Python-resident, so server-side traversal is moot.
- **Random-walk / node2vec structural embeddings.** We already have semantic (nomic) embeddings for membership; a second structural embedding adds a maintenance burden without a clear recall or connectivity win the bake-off didn't already get from PPR+PCST.

5 Gaps in the evaluation itself

Honest caveats about how much the bake-off actually proves:

- **Gold sets exist for only 3 of 5 bake-off topics** (diffusion, transformer, testtime); ssm and gnn report `recall=None`. Recall conclusions rest on three topics.
- **No LLM-judge / end-to-end compose** was run on the subgraph variants — the bake-off is retrieval-level only (recall, connectivity, drift). Whether better connectivity actually yields a better *review* is unmeasured. This is the natural next experiment if we want a defensible "why this method" for the writeup.
- m **is fixed at ≈40.** The recall/connectivity trade-off may move at different budgets; untested.
- **mean-cosine is a drift *proxy*, not a quality metric** — a method can score high by hugging the query and still miss the field's structure (exactly PCST's failure mode).

6 Recommendation

1. **Commit the synthesis as the phase-5 default**: PPR membership + PCST-on-landmarks connectivity + most-recent-predecessor structure. It is the empirically-justified best-of-both and already produces clean lineage trees across all five topics.
2. **Keep `induce` / `pcst` / `dsa_bfs` as config-toggleable selectors** (the ablation mechanism), so the phase-5 `run.py` can reproduce the bake-off and the writeup can cite a real comparison.
3. **Treat Group B as a separate, later, *offline* feature** (community/RAPTOR-style summaries for a browse UI), explicitly *not* on the retrieval hot path.
4. **Optional one probe before committing**: add SimGRAG as a fourth selector to close the "did you try everything in-regime?" question; expect it to underperform, then commit.
5. **The real missing experiment** is an LLM-judge pass over the subgraph variants (does connectivity → better review?), if we want the phase-5 method choice to be judged on the end task rather than on retrieval proxies alone.